

Level 3. Timing, reliability and safety

Level 1 ended with three questions about every arrow: how fresh, how much, and what if it stops. Level 2 drew the boxes for real robots. This level is about the arrows. By the end you should be able to put a time on every box and add them up, say what the robot does when a box goes quiet, and draw the state machine that decides what the robot is doing right now.

There is still almost no code; the one exception is five lines of C++ in section 3.3.

Rough time: two evenings.

3.1 Latency budgets

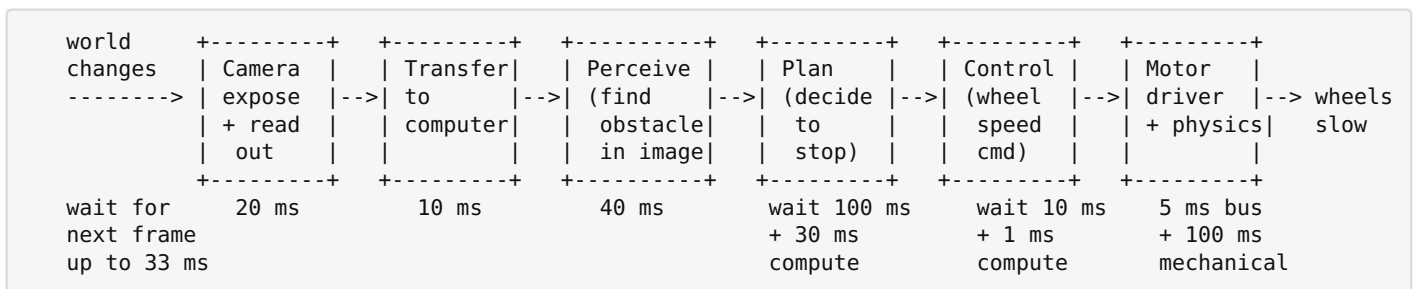
A story

A delivery robot is rolling along a pavement at 1 metre per second. A pushchair is parked 2 metres ahead. How long does the robot have?

The naive answer is "2 seconds". That is wrong twice over. First, the robot cannot stop instantly. If it brakes at 1 metre per second per second (a gentle stop that does not throw the parcels around), it travels half a metre while slowing down. Second, the robot does not know about the pushchair the instant it appears. The camera has to take a picture, the picture has to travel to the computer, a program has to find the pushchair in it, another program has to decide to stop, another has to tell the motors, and the motors have to actually slow the wheels. Every one of those steps takes time, and while they take time the robot keeps rolling at full speed.

The time from "the world changed" to "the wheels start to slow" is the **latency** of the whole chain. A **latency budget** is the list of every step in the chain with the worst time each one is allowed to take, added up, and compared with the time the physics gives you.

The chain, with a time on every box



Two kinds of cost appear on that drawing, and beginners always miss the first kind.

Waiting. The camera runs at 30 hertz (30 times a second, written 30 Hz), so it takes a picture every 33 milliseconds (ms). If the pushchair appears 1 ms after a picture was taken, the next picture is 32 ms away. The worst case is a full 33 ms of waiting before the camera even sees the obstacle. The same happens at every box that runs on its own timer. The planner runs at 10 Hz, so a new obstacle can wait up to 100 ms just for the planner's next tick, even if the planner's actual computation takes 30 ms.

Computing. The time the box spends actually working: running the detector, computing a path.

The budget table

Step	Worst-case time	Why
Wait for the next camera frame	33 ms	camera at 30 Hz, obstacle can appear just after a frame
Camera exposure and readout	20 ms	sensor integrates light, then ships pixels
Transfer to computer	10 ms	USB or Ethernet link
Perception (find the obstacle)	40 ms	neural network or point-cloud clustering
Wait for the next planner tick	100 ms	planner at 10 Hz
Planner compute	30 ms	replan the local path
Wait for the next control tick	10 ms	control loop at 100 Hz
Control compute	1 ms	PID and wheel speed maths
Motor driver and bus	5 ms	command travels over CAN bus to the driver board
Mechanical response	100 ms	motor current ramps, wheels actually start to slow
Total	349 ms	

So the robot travels at full speed for 0.349 seconds after the pushchair appears. At 1 m/s that is 0.35 metres. Then it brakes for another 0.5 metres. Total distance before it stops: 0.85 metres. The pushchair was 2 metres away, so there is 1.15 metres to spare.

Now change one number. Make the robot go 1.5 m/s. The reaction distance becomes 0.52 m and the braking distance becomes 1.13 m (braking distance grows with the square of speed). Total 1.65 m. Still under 2 m, just. At 2 m/s the total is 2.7 m and the robot hits the pushchair. The fastest speed that still stops in 2 m with this chain is about 1.68 m/s.

That is what a latency budget is for. It turns "is the robot fast enough" into a sum on paper, and it shows which box to speed up. The two biggest numbers here are the mechanical response (100 ms) and waiting for the planner (100 ms). Making perception twice as fast saves 20 ms. Running the planner at 20 Hz saves 50 ms. A direct "obstacle very close, stop now" arrow from perception to control, skipping the planner, saves 130 ms. Section 3.9 does that.

What happens when the budget is blown

The budget is a promise: every box finishes within its worst-case time. Boxes break the promise in ways that look harmless in a Python script:

- The perception node writes a big log file to disk and stalls for 500 ms.
- The planner is given a much bigger map than usual and takes 400 ms instead of 30.
- A garbage collector or memory allocator pauses the process for 50 ms at a bad moment.
- The network between two computers on the robot drops a packet and retries for 200 ms.

Take the first one. Perception takes 500 ms once instead of 40 ms. The chain becomes 809 ms. At 1 m/s the reaction distance is 0.81 m, total stopping distance 1.31 m, still under 2 m. At 1.5 m/s the reaction distance is 1.21 m, total 2.34 m, and the robot hits the pushchair. Same robot, same speed that was fine a minute ago, one slow frame.

Three lessons from that:

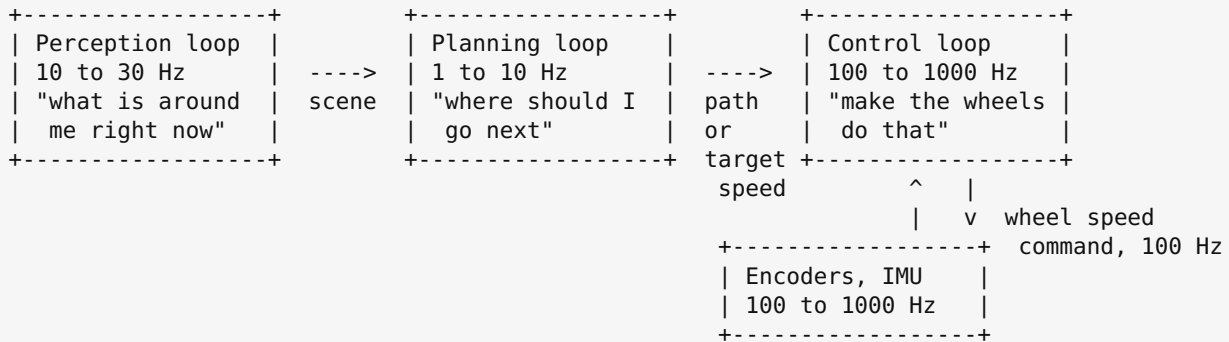
1. A latency budget is about the **worst** case, not the average. A box that is fast on average and slow once an hour is a box that crashes the robot once an hour.
2. The budget decides the **top speed**. If you cannot make the chain faster, you must make the robot slower or give it a sensor that sees further. That is a trade-off you choose on purpose: "top speed 1.2 m/s because stopping distance matters more than delivery time here".

3. Somebody must **watch** for blown budgets at run time, not just at design time. That is what the watchdogs in section 3.4 and the heartbeats in section 3.5 do.

3.2 Loops and rates

What a loop is

Most boxes in a robot run over and over on a timer. Such a box is called a **loop**, and how often it runs is its **rate**, in hertz. The time between two runs is the **period**: 100 Hz means a period of 10 ms, 1000 Hz means 1 ms. A robot has three kinds of loop, and they run at very different rates.



Why the rates are different

Each loop runs as fast as the thing it is fighting changes.

Control loop, 100 to 1000 Hz. It fights physics. A wheel motor's speed can change in a few milliseconds. A quadcopter tilts noticeably in 10 ms. If the control loop looked at the motor only ten times a second, the motor would wander a long way between looks and the correction would arrive late, which makes the robot oscillate. The PID controller in tutorial/06-robotics.md (b2) is this box; it wants a fresh measurement and a new command every period, with as little latency as possible.

Perception loop, 10 to 30 Hz. It fights the sensor. A camera gives 30 frames a second; a lidar spins at 10 or 20 Hz. Faster than the sensor gives nothing new to look at; slower throws frames away. So perception runs at the sensor's rate, or as fast as the computer keeps up.

Planning loop, 1 to 10 Hz. It fights the world, and the world changes slowly compared with a motor. A pedestrian walks about 1.4 m/s; replanning ten times a second means the plan is at most 14 cm stale. A global route across a city can be replanned once a second or only when a road closes. Planning is also expensive (the A* in tutorial/03-grids-bfs-dfs.md over a big grid can take tens of milliseconds), so you run it as rarely as you can get away with.

The trade-off sentence: "the control loop runs at 200 Hz because the motor's time constant is milliseconds and stability matters more than compute cost; the planner runs at 5 Hz because the world does not change faster and compute matters more than a fresher path".

How a slow box talks to a fast box

The control loop ticks every 10 ms. The planner gives it a new target speed every 100 ms. So nine out of every ten control ticks there is no new message. What does the control loop use?

It uses the **latest value**. The planner writes its answer into a slot; the control loop reads the slot every tick and does not care whether the value changed since last time. Think of a whiteboard: the planner writes on it whenever it has something new, the controller glances at it 100 times a second.

LATEST VALUE (a whiteboard)

QUEUE EVERY VALUE (a letterbox)

```

planner ---writes--> [ 0.8 m/s ] <--reads--- control    planner ---puts--> [msg][msg][msg][msg]...
10 Hz                one slot,      100 Hz          10 Hz
                    old value
                    overwritten
                                grows if the reader
                                is slower than the writer
                                --> reader takes from
                                    the front, in order

```

The other way round, a fast box feeding a slow box, is where beginners get hurt. Perception produces a frame at 30 Hz. Suppose the planner consumes frames at 10 Hz, and someone set the arrow up as a queue that keeps every message. Twenty frames a second pile up. After one minute the queue holds 1,200 frames. Each 640 by 480 colour frame is 0.92 megabytes, so that is 1.1 gigabytes of memory. Worse: the frame the planner is looking at after one minute was taken 40 seconds ago. The planner is steering around pedestrians who left long ago.

The rule:

Kind of data	Examples	Arrow style	Why
State: the current value of something	pose, image, lidar scan, battery level, target speed	keep the latest, depth 1	an old state is worthless; only "now" matters
Event: something that happened once	bump, goal reached, "start delivery", emergency stop	queue every one, bounded depth	losing an event means it never happened

Even the event queue must have a **depth** (a maximum length) and a rule for what happens when it is full: drop the oldest, drop the newest, or block the writer. Blocking the writer is the worst choice on a robot because it stalls the fast loop. Dropping the oldest is usual for state; for events you size the queue so it never fills, and log loudly if it does.

In ROS 2 (Robot Operating System 2, the middleware most companies use) this rule is called the **Quality of Service** or QoS setting of a topic: "keep last N" against "keep all", and "reliable" against "best effort". The idea has nothing to do with ROS. Any two loops joined by a buffer have a depth and a drop policy, whether you chose them or a library chose for you.

3.3 Threads and processes for a Python person

The two words

A **process** is a running program with its own private memory. `python planner.py` starts one. If it crashes, only it dies. Two processes cannot see each other's variables; to share data they must send it, over a pipe, a socket, or shared memory.

A **thread** is a worker inside a process. All threads of a process share the same memory: the same variables, the same vectors. Starting a thread is cheap and passing data between threads is free (just a pointer). The price is that two threads can touch the same variable at the same time, which is the bug in this section.

Python threads do not speed anything up because of the Global Interpreter Lock (GIL): only one thread runs Python code at a time. C++ has no GIL; two C++ threads on a four-core computer really do run at the same time. That is why the race below is a real bug in C++ and a rare one in Python.

One node per process, or many nodes in one process

In ROS 2 vocabulary a **node** is one box from your diagram, running as a program. You choose how to pack nodes into processes.

	One node per process	Many nodes in one process
A node crashes	only that node dies; a supervisor restarts it	the whole process dies, every node in it
Passing a 1 MB image	copied, or put in shared memory: 0.1 to 1 ms	pointer handed over: microseconds
Debugging	attach to one small program	one big program, harder to reason about
Typical use	development, non-critical nodes	camera plus perception pipeline, where copy cost matters

The trade-off sentence: "camera and perception share a process because copying 30 frames a second matters more than isolating them; the planner is its own process because being able to restart it after a crash matters more than the copy cost of a path".

Why the control loop gets its own thread

Inside a process, the control loop must tick every 10 ms no matter what. If it shares a thread with the planner, then while the planner is busy for 30 ms the control loop is not running, the motors get no new command, and the robot drifts. So the control loop is given a thread of its own, usually at a higher operating-system priority, and nothing else runs in it.

```
thread A (control, 100 Hz): tick . tick . tick . tick . tick . tick . tick . tick
thread B (planner, 10 Hz):  [===== plan 30 ms =====] . . . . . [===== plan =====]
                           ^ control keeps ticking while B is busy
```

The danger: two hands in one box

Thread B (planner) writes the new target speed into a variable. Thread A (control) reads it. A target speed is a small struct: linear speed and turn rate. Writing two numbers is two operations, and thread A can read in between.

```
shared box: target = { linear, angular }

thread B (writing 0.8, 0.5)      thread A (reading)
-----
writes linear  = 0.8
                               <--- A reads linear = 0.8, angular = 0.0 (the OLD one)
writes angular = 0.5

A now drives with a pair of numbers that never existed together.
```

This is a **race** (short for race condition): the result depends on which thread got there first. With a two-number struct the symptom is a twitch. With a `std::vector` being resized by one thread while another reads it, the symptom is a crash, or silent garbage, once a week.

Fix one: a lock

A **mutex** (mutual exclusion) is a token that only one thread can hold. Before touching the shared box you take the token; when you are done you give it back. A thread that finds the token taken waits. In C++ you never lock and unlock by hand; you make a `std::lock_guard`, which takes the mutex in its constructor and gives it back in its destructor. That is RAIL, the library-book-that-returns-itself idea from tutorial/02-cpp-interviewers-test.md (m3), and it means the mutex is released on every path, including exceptions and early returns.

```
std::mutex target_mutex;          // one token, shared by both threads
Target target;                   // the shared box
```

```
void set_target(Target t) {           // called by the planner thread
    std::lock_guard<std::mutex> hold(target_mutex); // take the token (constructor)
    target = t;                        // write both numbers together
}                                     // token returned here (destructor)
```

The read side does the same: lock, copy the struct out, unlock, then work on the copy.

The trade-off: locks are simple and correct, but the control thread can be **blocked**. If the planner holds the token for 2 ms because it is doing something silly inside the lock, the control loop's 10 ms tick is 2 ms late. Rule: hold a lock for a copy, never for a computation.

Fix two: a single-writer queue

Give the shared box a direction. Only thread B ever writes; only thread A ever reads; the data goes through a **queue** built so that a writer and a reader never need to wait for each other. For "latest value" data this is a one-slot mailbox that swaps a whole struct in one step. For events it is a lock-free ring buffer of fixed size.

```
planner (only writer) ---> [ slot ] ---> control (only reader)
                           swaps a whole Target at once; no half-written value can be seen
```

The trade-off: the reader never blocks, which is what a control loop wants, but the queue must be built carefully (or taken from a library) and it needs exactly one writer and one reader.

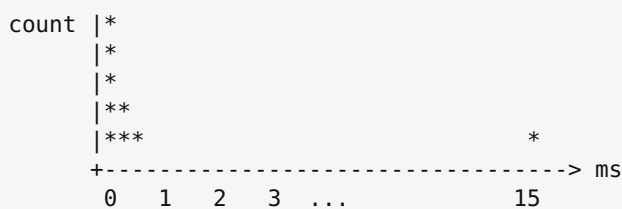
3.4 Real-time in plain words

Fast on average against always on time

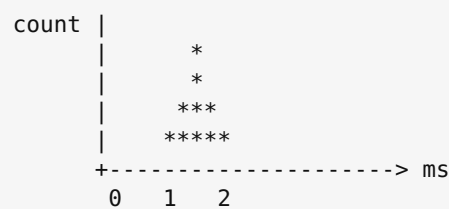
A **deadline** is the moment by which a box must have finished. The control loop at 100 Hz has a deadline every 10 ms. A **real-time** system is one where meeting the deadline is part of being correct: a control tick that arrives at 12 ms is not "a bit slow", it is wrong, because the motor has already drifted.

That is a different goal from "fast". Here are two versions of the same control step:

version 1: average 0.2 ms, worst ever 15 ms



version 2: average 0.9 ms, worst ever 1.1 ms



Version 1 is four times faster on average and useless in a 100 Hz control loop, because once in a while it takes 15 ms and misses its deadline. Version 2 is slower and always on time. For a control loop you want version 2. "Hard real-time" means a missed deadline is a failure (a surgical robot's arm controller, a drone's attitude loop). "Soft real-time" means a missed deadline degrades quality but is survivable (a video stream skipping a frame). Perception and planning are usually soft; control is usually hard.

What is banned inside the control loop

Everything with an unpredictable worst case is banned from the hard loop, even if its average is tiny.

Banned in the control loop	Typical average	Occasional worst case	Do this instead
Heap allocation (<code>new</code> , <code>push_back</code> past capacity, building a <code>std::string</code>)	50 to 500 ns	milliseconds (page fault, allocator lock)	allocate at start-up; <code>reserve()</code> once; use <code>std::array</code> (tutorial/02-cpp-interviewers-test.md, s5)
Taking a lock another thread might hold	20 ns	as long as the other thread holds it	single-writer queue (section 3.3)
Disk or file writes, <code>printf</code> to a console	10 us	10 to 100 ms	push a small record into a queue; a logging thread writes it
Network calls, waiting for a reply	100 us	seconds	never wait in the loop; read latest value
Anything that can throw or retry		unbounded	check inputs before the loop

The `reserve()` note in tutorial/02-cpp-interviewers-test.md (the memory section) and the `std::array` section (s5) are the coding side of this table. A control loop in C++ looks boring on purpose: fixed-size arrays, plain arithmetic, no allocation, no I/O.

A real-time operating system (RTOS), or Linux with the PREEMPT_RT patch, is the other half: it promises that when the 10 ms timer fires, the control thread gets the CPU within tens of microseconds instead of waiting behind a browser update.

Watchdogs

A **watchdog** is a timer that assumes the worst. The brain must reset it ("kick it") every so often, say every 100 ms. If the brain stops kicking, the watchdog fires and does something safe on its own: cuts motor power, engages brakes, or sends a zero-speed command. The watchdog lives in a place that does not depend on the brain: a small microcontroller on the motor driver board, or a hardware timer in the CPU that a crashed program cannot stop.

```
brain (control loop) --- "kick", every 10 ms ---> [ watchdog timer, 100 ms ]
                                     |
                                     no kick for 100 ms? ---> cut motor enable line
```

Numbers: with a 100 ms watchdog, a robot at 1 m/s rolls at most 0.1 m after its brain dies before the motors are cut. At 1.5 m/s with a lazier 300 ms watchdog it rolls 0.45 m. That distance is the price of the watchdog period, so the period is chosen from the physics, not from what is convenient for the software.

The trade-off: a short watchdog period stops the robot sooner but trips on harmless hiccups (a 10 ms watchdog on a loop that occasionally jitters to 11 ms will stop the robot for no reason). "100 ms because 10 cm of roll-on matters less than false stops every few minutes."

3.5 Failure modes

Level 1 asked, for every arrow, "what happens if it stops". Here is the list of things that actually stop, lie or break on a mobile robot, and the two questions for each: how do you notice, and what does the robot do.

What fails	How you detect it	What the robot does
Sensor goes silent (cable, driver crash)	no message for longer than N periods (a timeout on the arrow)	treat as "obstacle everywhere" for safety sensors; stop or slow; try to restart the driver
Sensor lies (lidar sees rain, camera blinded by sun, GPS reflects off buildings)	cross-check with another sensor; value jumps further than physics allows; quality flag from the driver	trust the other sensor more; slow down; if all sensors disagree, stop

What fails	How you detect it	What the robot does
Network drops between computers	heartbeat missing; packet counters; round-trip time rising	each side uses its last good value for a short time, then goes to a safe state
Node crashes	process exit code; missing heartbeat; supervisor notices	supervisor restarts it; if it crashes again within a minute, go to fault state
Battery low or dies	battery gauge below threshold; voltage sag under load	low: go home or finish the current job then stop; critical: stop where safe and call for help
Wheel slips (ice, gravel)	encoders say 1 m/s, IMU and lidar odometry say 0.3 m/s	stop trusting encoders for position; reduce speed; report the location
Motor driver over-temperature	driver reports it on the bus	reduce torque limit; stop before it shuts down on its own
Brain overloaded (CPU at 100 percent)	loop period measured longer than the deadline	shed load: drop perception rate, disable non-critical nodes, slow down
E-stop pressed	hardware line, no software involved	motors cut by hardware; software notices afterwards and enters fault state

Two patterns. Detection is almost always a **timeout** ("I expected a message and did not get one") or a **cross-check** ("two sources disagree"). The response is almost always a move towards a **safe state**.

The safe state

The **safe state** is the state the robot can go to from anywhere, without needing any of the things that might have broken, and in which it can wait for as long as it takes. For a pavement robot it is "stopped, brakes on, lights flashing, sending its position to the operations centre". For a drone it is "hover, then descend slowly", because "stop" is not available. For a surgical robot arm it is "hold position, do not release the tool", because letting go is the dangerous option. For a warehouse robot with a 500 kg shelf on its back it is "stop gently", because a hard stop tips the shelf over.

Choosing the safe state is the first design decision in reliability, and it is different for every robot. Everything else is about getting there.

Heartbeats

A **heartbeat** is a small message a box sends at a fixed rate that says nothing except "I am alive and it is now this time". Its only job is to be missed. Every box that another box depends on sends one; every box that depends on something listens for one and starts a timer.

```

perception ----- heartbeat, 10 Hz -----> supervisor
planner      ----- heartbeat, 10 Hz -----> supervisor
control      ----- heartbeat, 50 Hz -----> supervisor
remote ops   ----- heartbeat, 1 Hz -----> supervisor
supervisor: "no planner heartbeat
for 300 ms? -> command safe stop"
```

Three missed heartbeats at 10 Hz is 300 ms. A robot at 1.5 m/s covers 0.45 m in that time, so the number of misses you tolerate is, again, a physics decision. Heartbeats and watchdogs are the same idea at different layers: a watchdog is a heartbeat whose listener is hardware.

Degraded modes

Between "everything works" and "safe state" there is usually a ladder of **degraded modes**, each one doing less but still useful.

A **state machine** is a drawing with circles and arrows. Each circle is a **state**: a thing the robot can be doing, and in each state exactly one behaviour is active. Each arrow is a **transition**: an event that moves the robot from one state to another. The robot is in exactly one state at any moment, and that state is the answer to "what is the robot doing".

```
+-----+ new job +-----+ arrived +-----+ parcel +-----+ arrived
| IDLE |----->| NAVIGATING |----->| DELIVERING |----->| RETURNING |-----+
+-----+          +-----+ at house +-----+ taken +-----+ at hub |
      ^              ^               ^             ^
      |              |               |             |
road   road         crossing        road       crossing
ahead ahead         clear           ahead     clear
      v              |               v             |
      +-----+          +-----+          +-----+
      | WAITING AT |    (one state, drawn twice to keep | WAITING AT |
      | CROSSING   |    the arrows straight)          | CROSSING   |
      +-----+          +-----+          +-----+
+-----+-----+-----+-----+-----+-----+-----+
any state ---- "fault" (heartbeat lost, e-stop, battery critical) ----> +-----+
                                   | FAULT | --- "operator cleared" ---> IDLE
                                   +-----+
```

From state	Event	To state	What happens on the way
DELIVERING	10 minutes with no customer	RETURNING	lock the lid; flag "not delivered"
RETURNING	road ahead	WAITING AT CROSSING	as above
RETURNING	arrived at hub	IDLE	disable the planner; start charging
any state	fault	FAULT	command safe stop; lights on; report to operations
FAULT	operator cleared	IDLE	re-run self-checks

The rules that make a state machine safe are simple:

- Every state has a way to FAULT, and FAULT needs nothing that might be broken.
- Every transition is logged with a timestamp (section 3.8 depends on this).
- Entering a state resets whatever that state needs; leaving a state cleans up. The "what happens on the way" column is where that lives.
- No state depends on remembering how it was reached. If it does, you are missing a state.

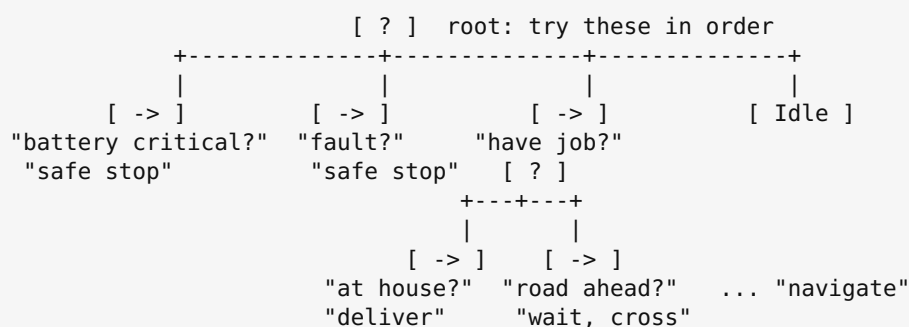
The supervisor ticks slowly, 5 to 10 Hz. It computes no paths and no wheel speeds; it switches other boxes on and off and hands them goals.

When a behaviour tree is better

State machines have a scaling problem. Add a "battery low" check and you need a transition from every state. Add "customer cancelled" and you need another from every state. Ten states and five such conditions is fifty arrows. The table becomes a wall.

A **behaviour tree** is a different way to write the same decisions. It is a tree that is ticked from the root, several times a second. Each node returns one of three answers: success, failure, or running. Two kinds of inner node do the work:

- A **sequence** (drawn `->`) runs its children left to right and stops at the first that fails. "Do A, then B, then C."
- A **fallback** or selector (drawn `?`) runs its children left to right and stops at the first that succeeds. "Try A; if that fails try B; if that fails try C."



Every tick the root asks, left to right: is the battery critical? If yes, safe stop and stop looking. Else, is there a fault? Else, do I have a job? The battery check is written once and applies in every situation, because it is to the left of everything else. That is the thing state machines could not do cheaply.

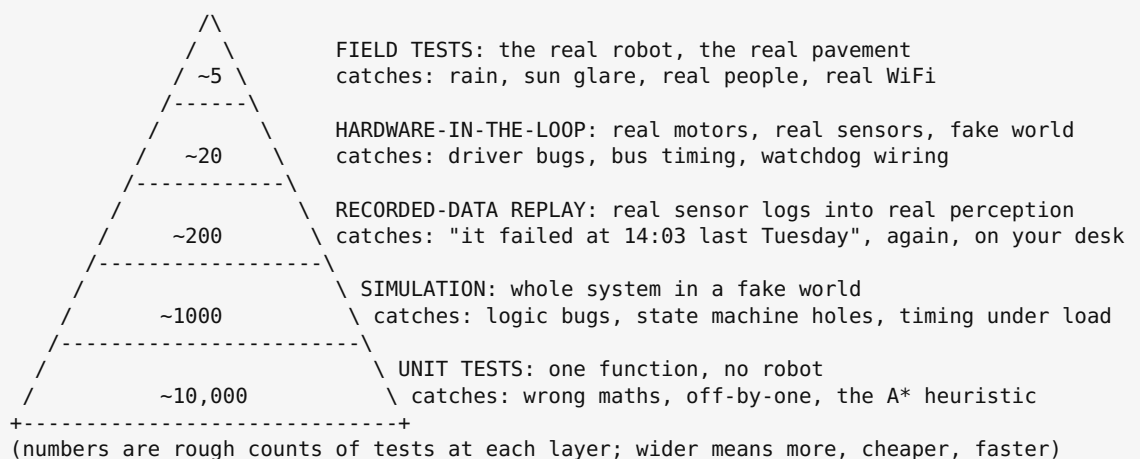
	State machine	Behaviour tree
Best for	a few clear modes with clean hand-overs (delivery robot, vacuum, drone flight phases)	many behaviours that share the same guards (battery, fault, cancel)

	State machine	Behaviour tree
Answer to "what is the robot doing"	the current state, one word	the path from root to the running leaf
Adding a global check	an arrow from every state	one node on the left
Reading it	the table is a complete list of what can happen	you must read tick order to know what wins
Debugging	log the state; easy	log the tree's running path; more to read

The trade-off sentence: "a state machine because the robot has six clear modes and an operations team who need to read the current mode on a dashboard; a behaviour tree would only pay off if the modes multiplied". Many real systems use both: a small state machine on top (idle, working, fault), and a behaviour tree inside "working".

3.7 Testing a robot system

You cannot test a robot by driving it into a pushchair. You can test each layer of the chain in the place where that layer's bugs live cheaply.



Unit tests. One function, known input, expected output, runs in milliseconds. The A* project at `code/03-grids-bfs-dfs/astar` has a `tests/astar_test.cpp` built with GoogleTest: it feeds small grids to the planner and checks the path length and that the path never crosses a wall. That is exactly what a planner box's unit tests look like on a job. Perception boxes are unit-tested with a handful of saved images; controllers with a simulated plant, as in the PID section of `tutorial/06-robotics.md` (b2), which ships with its own tiny simulation.

Simulation. Every box runs for real; only the sensors and motors are replaced by a physics program (Gazebo and Isaac Sim are two common ones). It catches bugs that live between boxes: the state machine never leaves WAITING AT CROSSING; the planner and controller disagree about which way is left; the queue from section 3.2 fills up. It also lets you run a thousand deliveries overnight, with random pedestrians, which the field could never give you. What it does not catch: anything the simulator got wrong, which is always the sensors.

Recorded-data replay. Record every message on every arrow while the real robot drives, with timestamps (in ROS 2 this recording is a **bag**, from the old name rosbag). Later, play the recording back into the perception and planning boxes as if the robot were there. When the robot did something strange at 14:03, you replay 14:02 to 14:04 on your desk, change the code, replay again. Replay also turns one real drive into

a regression test: "the new detector must find every pedestrian the old one found in the 200 hours of bags we have". It cannot test control (the recorded wheels do not respond to your new commands), and it only ever shows you what the robot already saw.

Hardware-in-the-loop (HIL). The real computer, real motor drivers and real sensors, but the world is fake: the motors spin on a bench, the lidar looks at a screen or the simulator injects scans directly at the driver level. It catches the things simulation misses because they are about hardware: the CAN bus stalls when the log rate is high, the watchdog is wired to the wrong pin, the real camera has 60 ms of latency where the simulated one had zero.

Field tests. The real robot on the real pavement, with a person walking behind it holding an emergency stop. It catches what nothing else can: sun glare at 4 pm in October, a dog, WiFi dropping behind one particular building. It is slow and expensive, so everything that could be caught lower in the pyramid should have been.

The trade-off is cost against realism, and the design rule is: push every bug as far down the pyramid as it will go. A bug found in the field should end its life as a unit test or a replay test, so that it never needs the field again.

3.8 Logging and observability

Observability means: when the robot did something, can you find out why afterwards, from what it wrote down. A robot that stops in the street and leaves no trace has failed twice.

What to log, and at what rate

Logging everything is not free. What the delivery robot's sensors produce:

Stream	Size per message	Rate	Per hour	8-hour shift
2D lidar scan (1,080 ranges, 4 bytes each)	4.3 KB	10 Hz	156 MB	1.2 GB
Camera, raw 640 by 480 colour	0.92 MB	15 Hz	50 GB	400 GB
Camera, JPEG compressed (about 50 KB per frame)	50 KB	15 Hz	2.7 GB	22 GB
Odometry (pose, speed, timestamps; about 100 bytes)	100 B	50 Hz	18 MB	0.14 GB
State machine transitions	100 B	on change	KB	KB
Heartbeats and loop timing summaries	50 B	1 Hz per box	MB	MB

Raw camera is out of the question: a shift would fill a laptop. So the rule is layered:

1. **Always, everywhere, cheaply:** every state machine transition, every fault, every timeout, every dropped message, every loop that missed its deadline, with a timestamp and the name of the box. This is tiny and it answers most questions.
2. **Always, at reduced rate:** pose, target speed, battery, lidar scans. Enough to replay what the robot thought was around it.
3. **On trigger only:** the last 30 seconds of compressed camera frames, kept in a ring buffer in memory and written to disk only when a fault, an e-stop, or a hard brake happens. You get the pictures for the interesting moments and nothing for the boring ones.

Everything is stamped with the same clock. If two computers on the robot disagree by half a second, the log is a puzzle instead of a story, so clock sync is part of the design.

"Why did the robot stop at 14:03?"

An operator asks. Here is what the log looks like when the design above has been followed:

```

14:03:11.204 supervisor state NAVIGATING -> FAULT reason: heartbeat timeout: perception
14:03:11.203 supervisor heartbeat missing: perception, last seen 14:03:10.901 (302 ms)
14:03:10.950 control target speed 1.2 -> 0.0 (safe stop commanded by supervisor)
14:03:10.901 perception heartbeat ok, frame 41877, detect 38 ms
14:03:10.860 perception warning: detect took 210 ms (deadline 60 ms), frame 41876
14:03:10.640 perception warning: detect took 190 ms (deadline 60 ms), frame 41875
14:03:10.410 system CPU 98 percent, top: log_writer 61 percent
14:03:10.400 log_writer flushing camera ring buffer (32 MB) after hard-brake trigger at 14:03:09.9
14:03:09.910 control hard brake: decel 2.1 m/s^2 (pedestrian at 0.9 m)

```

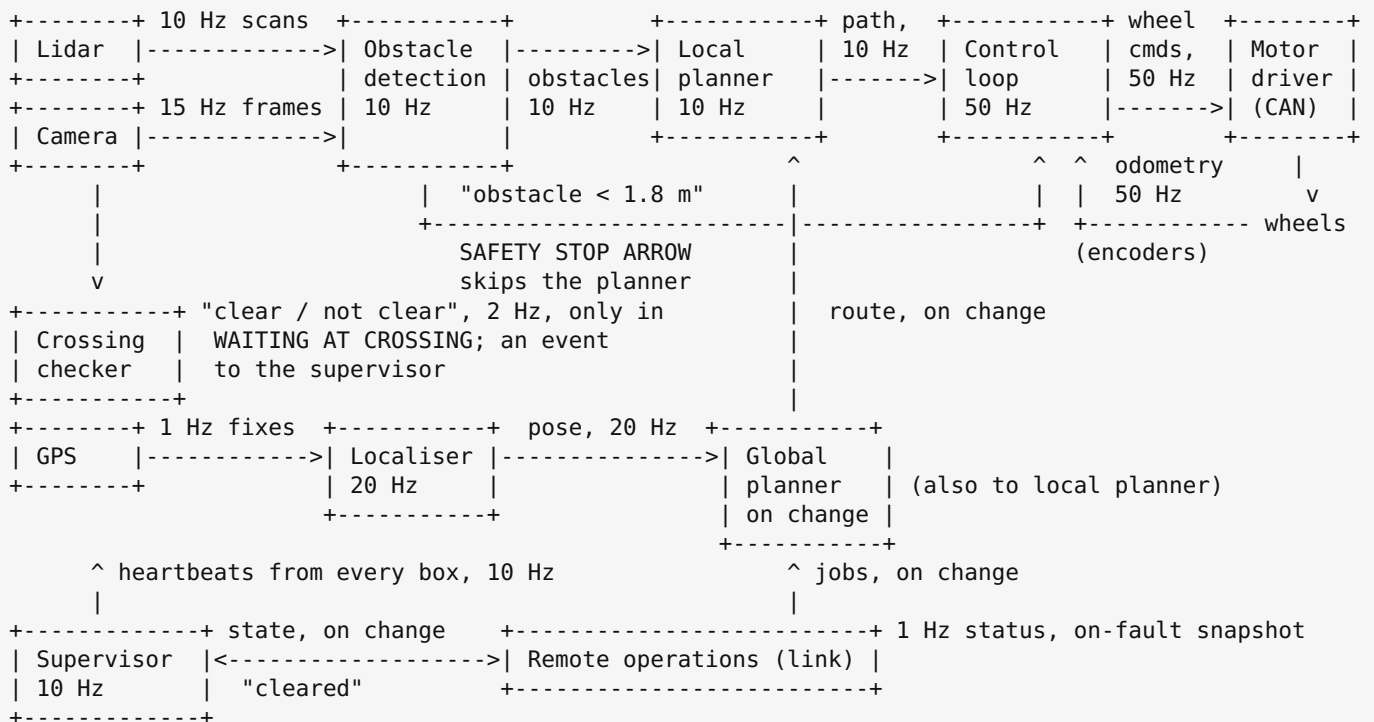
Read from the bottom. A pedestrian stepped out; the robot braked hard (good); the hard brake triggered the camera dump (as designed); the dump ate the CPU; perception missed two deadlines and then a heartbeat; the supervisor went to FAULT as told. Every line came from rule 1 above. The fix is obvious once you can see it: the log writer needs a lower priority than perception. Without the loop-timing warnings and the CPU line you would only know "perception heartbeat lost" and would spend a day blaming the camera driver.

That is observability: enough breadcrumbs, cheaply, that a stop has a story.

3.9 Worked example: the delivery robot, with time and failure on it

Take the Level 2 delivery robot (pavement, shop to house, lidar plus camera plus GPS, a remote operations centre on a mobile link). Level 2 drew its boxes. Now put numbers and failure behaviour on the arrows.

The boxes and their loops



Loop	Rate	Period	Why this rate
Control	50 Hz	20 ms	wheel motors respond in tens of ms; 50 Hz is enough for a slow heavy robot and leaves CPU for perception

Loop	Rate	Period	Why this rate
Obstacle detection	10 Hz	100 ms	the lidar spins at 10 Hz; running faster shows the same scan twice
Local planner	10 Hz	100 ms	pedestrians move 14 cm between plans; enough, and A* over the local grid costs about 20 ms
Localiser	20 Hz	50 ms	fuses odometry (50 Hz) with GPS (1 Hz); 20 Hz keeps the pose fresh for the planner
Global planner	on change		a route across the town only changes when the job or a road closure does
Supervisor	10 Hz	100 ms	checks heartbeats and events; nothing it does needs to be faster
Crossing checker	2 Hz	500 ms	only while stopped at a kerb; a traffic light changes over seconds

Arrows carry "latest value" (scans, frames, pose, path, wheel commands) except three that are queued events: jobs from operations, "cleared" from the operator, and "obstacle < 1.8 m" on the safety arrow, which is an event with a queue depth of 1 (a newer stop overrides an older one, and one is enough).

The latency budget, twice

The normal path, world change to wheels slowing:

Step	Worst case
Wait for the next lidar scan (10 Hz)	100 ms
Lidar transmits the scan	10 ms
Obstacle detection	20 ms
Wait for the next local planner tick (10 Hz)	100 ms
Local planner compute	20 ms
Wait for the next control tick (50 Hz)	20 ms
Control compute	1 ms
Motor driver and CAN bus	10 ms
Mechanical response	150 ms
Total	431 ms

The safety-stop path, which skips the planner:

Step	Worst case
Wait for the next lidar scan	100 ms
Lidar transmits the scan	10 ms
Safety zone check (is any point inside 1.8 m ahead)	2 ms
Wait for the next control tick	20 ms
Control compute	1 ms
Motor driver and CAN bus	10 ms
Mechanical response	150 ms

Step	Worst case
Total	293 ms

Braking at 1 m/s per second (parcels and pedestrians both prefer it), the stopping distances and the fastest safe speed for a given clear distance are:

Speed	Reaction, normal path	Braking distance	Total, normal	Total, safety path
0.5 m/s	0.22 m	0.13 m	0.34 m	0.27 m
1.0 m/s	0.43 m	0.50 m	0.93 m	0.79 m
1.5 m/s	0.65 m	1.13 m	1.77 m	1.57 m

Clear distance ahead	Max speed, normal path	Max speed, safety path
1 m (someone steps out of a doorway)	1.05 m/s	1.15 m/s
2 m (a parked pushchair on a narrow pavement)	1.61 m/s	1.73 m/s
5 m (a wide, open pavement)	2.76 m/s	2.88 m/s

That last table is the design. It says the robot's top speed is not a number but a rule: drive at 1.5 m/s when the lidar shows 2 m or more of clear pavement, and drop to 1 m/s next to doorways and around corners. It also shows that the safety arrow buys only about 0.1 m/s, because the lidar's 100 ms wait and the 150 ms of mechanics dominate both paths. The honest trade-off: "the safety arrow is worth having because it also works when the planner has crashed, not because it is much faster; if we wanted real speed we would buy a 20 Hz lidar, which halves the biggest number in both tables".

Failure handling on this robot

Failure	Detected by	Robot does	Mode
Lidar silent for 300 ms	timeout in obstacle detection	safety stop; supervisor to FAULT	safe state
Camera silent	timeout in detection	continue on lidar at 1.0 m/s; no autonomous crossings	degraded 1
GPS silent or jumping more than 5 m	localiser cross-check with odometry	continue on odometry for 60 s at 0.5 m/s, then stop	degraded 2
Local planner heartbeat lost (300 ms)	supervisor	safety stop; restart the planner; if it dies again within a minute, FAULT	safe state
Control loop heartbeat lost	motor driver watchdog, 100 ms	hardware cuts motor enable; brakes hold	safe state
Mobile link to operations lost	1 Hz heartbeat missed 10 times	continue the current delivery autonomously; do not cross roads; FAULT if still lost after 10 minutes	degraded 1
Battery under 20 percent	gauge	finish the current delivery, return, no new jobs (20 percent of a 4-hour pack is 48 minutes)	degraded
Battery under 5 percent	gauge	stop at the next safe spot; call for pickup	safe state

Failure	Detected by	Robot does	Mode
Wheel slip	encoders against lidar odometry disagree by more than 30 percent	halve speed; stop trusting encoders in the localiser	degraded
E-stop	hardware	motors cut; supervisor to FAULT when it notices	safe state

Numbers behind two rows: three missed 10 Hz heartbeats is 300 ms, and at 1.5 m/s the robot rolls 0.45 m before the safety stop begins, which is why the tolerance is three misses and not ten. The 100 ms hardware watchdog on the control loop costs at most 0.15 m at 1.5 m/s.

The supervisor

The state machine of section 3.6 is this robot's supervisor, plus one thing from the degraded-modes ladder: every state carries a **mode** flag (full, degraded 1, degraded 2) that the speed limit and the crossing checker read. The mode is a flag, not a state, because the robot is still navigating when a camera dies. The trade-off: "a flag because six states times three modes is eighteen states, and the operations dashboard should still read NAVIGATING".

What changed from Level 2

Level 2 drew the same boxes. This level added a rate on every loop, a worst-case time on every box, two budgets turned into a speed rule, a safety arrow that bypasses the planner, heartbeats, a hardware watchdog, a failure table with a safe state and a degraded ladder, and a state machine with FAULT reachable from everywhere. None of it needed code. All of it is the difference between a diagram and a design.

3.10 Try it, in your own words, check yourself

Try it

1. Draw the latency chain for a robot arm that must stop when a hand enters its workspace, watched by a 3D camera at 15 Hz. Put a worst-case time on each box, add them up, and work out how far a 1 m/s arm tip travels before it stops at 5 m/s per second of braking. Is a light curtain (a beam sensor wired straight to motor power) worth adding? What does it do to the chain?
2. Write the failure table for the drone inspecting a wind turbine blade from Level 1's "Try it". Start with the safe state; it is not "stop".
3. Draw the state machine for the bin-picking arm (idle, waiting for image, planning grasp, moving to pick, gripping, moving to place, releasing, fault). Which transitions need a timeout? Then rewrite it as a behaviour tree and decide which drawing you would rather maintain, and why.
4. Run the GoogleTest suite in code/03-grids-bfs-dfs/astar. Add one test for a planner bug you can imagine (the path leaves the grid, say). Which layer of the pyramid did you add to?
5. Change the lidar in section 3.9 to 20 Hz and the mechanics to 80 ms; redo the speed table.

In your own words

Every box takes time, and waiting for a box's next tick is time too. Add the worst cases along the chain, compare with what the physics allows, and you get the robot's top speed. Loops run at the rate of the thing they fight: control at hundreds of hertz for the motors, perception at the sensor's rate, planning at a few hertz for the world. Fast boxes read the latest value from slow boxes; events are queued, with a depth and a drop rule. Threads share memory and therefore need a lock or a single-writer queue; the control loop gets its own thread and avoids anything with an unpredictable worst case, and a hardware watchdog stops the

motors if that thread goes quiet. Every arrow has a timeout and every robot has a safe state, with degraded modes on the way to it. A state machine says what the robot is doing now; a behaviour tree is better when many behaviours share the same guards. Test low in the pyramid, log every transition and every missed deadline, and every stop will have a story.

Check yourself

1. A camera runs at 20 Hz and the box after it runs at 5 Hz. What is the worst-case waiting time added by those two boxes before any computation happens?
2. A robot at 2 m/s has a 400 ms chain and brakes at 2 m/s per second. How far does it travel before stopping? Show the two parts of the answer.
3. Why does the control loop run at 100 Hz or more while the planner runs at 5 Hz? Give the physical reason for each.
4. Perception produces poses at 30 Hz; the logger consumes them at 10 Hz through a queue that keeps every message. Describe what is wrong after two minutes, in memory and in latency.
5. Give one example of state data and one example of event data on a warehouse robot, and say which arrow style each one gets.
6. Two threads share a struct with two fields. Describe the race in one sentence and name the two fixes. Which one would you pick for the control loop's input, and why?
7. What does "real-time" promise that "fast" does not? Name three things banned from a hard control loop and what to do instead of each.
8. Where does a watchdog live, and why there? How would you choose its period for a 2 m/s forklift robot?
9. Name the safe state for a pavement robot, a drone and a surgical arm, and say why they differ.
10. An operator asks why the robot stopped at 14:03. List the four kinds of log line you would expect to find, in the order you would read them.